

PYTHON FILE HANDLING

1. INTRODUCTION TO FILE HANDLING

Definition

File handling in Python refers to the process of creating, reading, writing, appending, and managing files stored on secondary storage devices such as hard disks. Files allow programs to store data permanently, unlike variables which store data temporarily in memory.

Why File Handling is Needed

- To store data permanently
- To handle large amounts of data
- To share data between programs
- To maintain logs and records

Example

```
file = open("data.txt", "w")  
file.write("Hello File Handling")  
file.close()
```

2. TYPES OF FILES IN PYTHON

Text Files

Text files store data in human-readable form. Example: .txt, .csv

Binary Files

Binary files store data in binary format. Example: .jpg, .pdf, .dat

Example

```
file = open("sample.txt", "r")  
print(file.read())  
file.close()
```

3. OPENING A FILE

Definition

The `open()` function is used to open a file in Python.

Syntax

```
file_object = open(file_name, mode)
```

Example

```
file = open("example.txt", "w")
```

4. FILE MODES

Read Mode (r)

Used to read an existing file.

Write Mode (w)

Creates a new file or overwrites an existing file.

Append Mode (a)

Adds data to the end of a file.

Read and Write Mode (r+)

Allows reading and writing.

Example

```
file = open("data.txt", "a")
```

```
file.write("New Line")
```

```
file.close()
```

5. WRITING TO A FILE

Definition

Writing means storing data into a file.

`write()` Method

Writes a single string to a file.

`writelines()` Method

Writes multiple strings to a file.

Example

```
file = open("info.txt", "w")  
file.write("Python File Handling")  
file.close()
```

6. READING FROM A FILE

read()

Reads entire file content.

readline()

Reads one line at a time.

readlines()

Reads all lines into a list.

Example

```
file = open("info.txt", "r")  
print(file.read())  
file.close()
```

7. APPENDING DATA TO A FILE

Definition

Appending means adding data without removing existing content.

Syntax

```
file = open("info.txt", "a")
```

Example

```
file = open("info.txt", "a")  
file.write("\nNew Content")  
file.close()
```

8. FILE POINTER

Definition

The file pointer indicates the current position in a file.

tell()

Returns current position of file pointer.

seek()

Changes file pointer position.

Example

```
file = open("info.txt", "r")  
print(file.tell())  
file.seek(0)  
file.close()
```

9. CLOSING A FILE

Definition

Closing a file frees system resources.

close() Method

```
file.close()
```

10. WITH STATEMENT

Definition

The with statement automatically closes the file.

Syntax

```
with open("data.txt", "r") as file:  
    print(file.read())
```

11. FILE EXCEPTIONS

Common Exceptions

- FileNotFoundError
- PermissionError
- IOError

Example

try:

```
file = open("no_file.txt", "r")
```

except FileNotFoundError:

```
print("File not found")
```

12. COPYING FILE CONTENT

Example

with open("source.txt", "r") as src:

with open("dest.txt", "w") as dest:

```
dest.write(src.read())
```

13. COUNTING LINES, WORDS, CHARACTERS

Example

```
file = open("data.txt", "r")
```

```
text = file.read()
```

```
print(len(text.split()))
```

```
file.close()
```

14. SEARCHING DATA IN FILE

Example

```
word = "Python"
```

```
file = open("data.txt", "r")
```

```
if word in file.read():
```

```
    print("Found")

else:

    print("Not Found")

file.close()
```

15. SUMMARY

File handling is an essential concept in Python that allows programs to store and manage data efficiently. Understanding file modes, methods, and exception handling is crucial for real-world applications.

16. FILE HANDLING USING OS MODULE

Definition

The `os` module in Python provides functions to interact with the operating system, especially for file and directory operations such as creating, deleting, and renaming files.

Common Functions

- `os.rename()` – Renames a file
- `os.remove()` – Deletes a file
- `os.path.exists()` – Checks if file exists

Example

```
import os

if os.path.exists("data.txt"):

    os.rename("data.txt", "newdata.txt")
```

17. FILE HANDLING USING SHUTIL MODULE

Definition

The `shutil` module is used for high-level file operations such as copying and moving files.

Common Functions

- `shutil.copy()`
- `shutil.move()`

Example

```
import shutil

shutil.copy("source.txt", "backup.txt")
```

18. WORKING WITH CSV FILES

Definition

CSV (Comma Separated Values) files store tabular data in plain text format.

CSV Module

Python provides the `csv` module to handle CSV files easily.

Example

```
import csv

with open("data.csv", "w", newline="") as file:

    writer = csv.writer(file)

    writer.writerow(["Name", "Age"])

    writer.writerow(["Krish", 21])
```

19. WORKING WITH BINARY FILES

Definition

Binary files store data in binary format and are used for images, audio, and video files.

Example

```
file = open("image.jpg", "rb")

data = file.read()

file.close()
```

20. FILE BUFFERING

Definition

Buffering refers to temporarily storing data in memory before writing it to a file to improve performance.

Example

```
file = open("data.txt", "w", buffering=1)
file.write("Buffered Data")
file.close()
```

21. FILE HANDLING BEST PRACTICES

Points

- Always close files after use
 - Use with statement
 - Handle exceptions properly
 - Avoid hardcoded file paths
-

22. REAL-TIME APPLICATIONS OF FILE HANDLING

Examples

- Student management systems
 - Banking transaction logs
 - Data backup systems
 - Report generation
-

23. DIFFERENCE BETWEEN FILE HANDLING AND DATABASE

File Handling	Database
Simple storage	Structured storage
Less secure	More secure
No query support	Supports SQL

24. COMMON MISTAKES IN FILE HANDLING

Mistakes

- Forgetting to close files
 - Using wrong file mode
 - Not handling exceptions
-

25. CONCLUSION

Python file handling provides an efficient way to store and manage persistent data. By mastering file operations, modes, exception handling, and advanced modules like `os`, `shutil`, and `csv`, programmers can build robust and real-world applications.